Project Outline and Motivation 3
Updated Notes

Alon Sportes

1 What problem are we solving?

The project solves the one-dimensional time-independent Schrödinger equation. In the dimensionless convention used in the code, it is written as

$$\psi''(x) = 2\,[V(x) - E]\psi(x). \tag{1}$$

The unknowns are:

- $\psi(x)$: the wavefunction,
- E: the energy eigenvalue,
- $V(x)$: the chosen potential.

This is not just an ordinary differential equation problem. It is a boundary-value eigenvalue problem. Only special values of E give physically valid bound states. This matches the proposal and the report structure exactly.

2 Which course topics are used?

2.1 Lecture 1: Numerical errors and computational project expectations

This project uses the main idea from Lecture 1: numerical computation is approximate, so numerical errors must be controlled and the method must be documented. The final project instructions also require code documentation and a short writeup explaining the problem, the method, the results, and the convergence and correctness checks.

In this project, this appears through:

- convergence studies,
- comparison with exact solutions,
- normalization checks,
- automated tests,
- README documentation,

- discussion of truncation error, round-off error, and numerical floors.

A very important example is the infinite square well convergence study. The solver originally produced very accurate energies, but the observed convergence order was not ideal. The issue was not the Numerov recurrence itself, but the startup values used to seed the two-step recurrence. After adding higher Taylor terms to the parity-based startup conditions, the square-well convergence slopes improved to approximately fourth order. This is exactly the kind of numerical-error diagnosis emphasized in the course.

2.2 Lecture 3: Numerical differentiation and integration

Numerical integration is used when normalizing the wavefunction:

$$\|\psi\| = \sqrt{\int |\psi(x)|^2 \, dx}. \tag{2}$$

In the code, this is done with `np.trapezoid`. The reason is physical: quantum wavefunctions must satisfy

$$\int |\psi(x)|^2 \, dx = 1. \tag{3}$$

Therefore, normalization is not optional. It makes the solution physically meaningful.

Numerical differentiation is also used in `derivative_at_right_edge()`. This is especially important for inward shooting. For even states, the condition at the origin is

$$\psi'(0) = 0. \tag{4}$$

If the derivative is computed with a low-order stencil, then the whole eigenvalue calculation can lose accuracy even if the Numerov recurrence itself is high order. This happened in the harmonic oscillator case. The code now uses a fourth-order backward stencil when enough points are available. This keeps the boundary-condition accuracy consistent with the rest of the method.

2.3 Lecture 4: Root finding

Root finding is used to determine the allowed energies. The project uses bisection: first detect a sign change, then repeatedly cut the interval in half.

In the code this appears in:

- `find_brackets()`,
- `bisect_energy()`,
- `find_inward_decay_brackets()`,
- `bisect_energy_inward_decay()`,
- `find_rk4_brackets()`,

- `bisect_rk4_energy()`.

Bisection is used because it is stable and easy to justify. Newton's method is faster, but it requires derivatives and can fail if the starting guess is poor. Bisection is slower, but it is robust.

The project also includes root-finding diagnostics. These are not strictly required, but they are useful because they show how the mismatch function crosses zero and how the bisection process isolates eigenvalues.

2.4 Lecture 5: ODEs, boundary-value problems, eigenvalue problems, and shooting

This is the core lecture for the project. The Schrödinger equation is a second-order ODE, but unlike an initial-value problem, the correct energy is not known in advance. Therefore, the method is:

1. guess an energy E,
2. integrate the ODE,
3. check the boundary condition,
4. improve the energy guess.

That is the shooting method.

The project uses two shooting strategies:

- outward shooting from $x = 0$,
- inward shooting from $x_{\max}$ to $x = 0$.

Outward shooting is used for many symmetric finite-domain or effectively finite-domain problems. Inward shooting is used mainly for the harmonic oscillator. This is because the harmonic oscillator is defined on an infinite domain, and outward integration can pick up the unphysical growing exponential tail. Inward shooting starts from the decaying asymptotic side and imposes the parity condition at the origin.

2.5 Lecture 6: PDEs and Schrödinger equation context

Lecture 6 mentions the Schrödinger equation as one of the important PDEs in physics. This project uses the time-independent Schrödinger equation in one dimension, which becomes an ODE eigenvalue problem. Therefore, this project is not solving a time-dependent PDE. It is using an ODE method on the stationary quantum problem.

2.6 What is not used

The project does not use:

- finite-volume methods,

- upwind methods,
- CFL conditions,
- Riemann solvers,
- shock capturing,
- slope limiters,
- Euler fluid equations,
- MHD,
- Particle in Cell.

Those topics belong to the advection, hydrodynamics, Burgers equation, Euler equations, and PIC parts of the course. They are not part of this project.

3 Why Numerov?

The equation has the form

$$\psi''(x) = q(x)\psi(x), \tag{5}$$

where

$$q(x) = 2[V(x) - E]. \tag{6}$$

This form is exactly what Numerov is designed for: second-order ODEs without a first-derivative term.

In `src/numerov.py`:

- `q_from_energy()` builds $q(x)$,
- `numerov_outward()` performs the Numerov recurrence,
- `normalize_wavefunction()` rescales the final wavefunction so that total probability is 1,
- `derivative_at_right_edge()` estimates the derivative at the boundary when needed.

Numerov is used instead of Euler or a generic Runge-Kutta method because it is especially efficient for equations like the Schrödinger equation. It uses neighboring grid values and gives high accuracy for second-order linear ODEs. This is also consistent with Pang, where the one-dimensional Schrödinger equation is treated using wavefunction integration and eigenvalue search.

A key implementation detail is that Numerov is a two-step method. It needs ψ at the first two grid points. Therefore, the code must provide startup values. In `shooting.py`, `initial_conditions()` gives those startup values using Taylor expansion and parity.

For even states:

$$\psi(0) = 1, \qquad \psi'(0) = 0. \tag{7}$$

For odd states:

$$\psi(0) = 0, \qquad \psi'(0) = 1. \tag{8}$$

The latest code includes higher Taylor terms:

- even states include the h^4 term,
- odd states include the h^5 term.

This matters because if the startup formula is too low order, it can pollute the observed convergence rate. This was seen in the infinite square well: the energies were accurate, but the convergence slopes were not cleanly fourth order. After the startup correction, the first square-well states show slopes close to 4.

4 Why shooting?

A bound state must behave correctly at both boundaries. However, Numerov integration needs starting values. Therefore, the algorithm is:

1. choose a trial energy E,
2. choose initial conditions at $x = 0$, or from the decaying tail,
3. integrate with Numerov,
4. check whether the wavefunction satisfies the target boundary condition,
5. adjust E.

This is shooting.

In `src/shooting.py`, the outward shooting logic is built from:

- `initial_conditions()`,
- `half_domain_wavefunction()`,
- `boundary_mismatch()`,
- `find_brackets()`,
- `bisect_energy()`,
- `solve_state_from_bracket()`,
- `solve_symmetric_potential()`.

The important function is `boundary_mismatch()`. It answers the question: for this trial energy, did the wavefunction land correctly at the boundary? If not, the energy is wrong.

The inward shooting logic is built from:

- `inward_decay_initial_conditions()`,
- `inward_decay_half_domain_wavefunction()`,
- `inward_decay_boundary_mismatch()`,
- `find_inward_decay_brackets()`,

- `bisect_energy_inward_decay()`,
- `solve_symmetric_potential_inward_decay()`.

Inward shooting is used for confining potentials such as the harmonic oscillator. The physical solution decays at large x. If we integrate outward from the origin, the numerical solution can pick up the unphysical growing mode. Inward shooting starts in the forbidden region and integrates toward the origin, giving cleaner harmonic oscillator wavefunctions and more reliable parity enforcement.

5 Why parity?

The main bound-state potentials are symmetric:

$$V(-x) = V(x). \tag{9}$$

For symmetric potentials, quantum states are either even or odd.

Even states satisfy

$$\psi(-x) = \psi(x), \tag{10}$$

and odd states satisfy

$$\psi(-x) = -\psi(x). \tag{11}$$

Therefore, instead of integrating from $-x_{\max}$ to $x_{\max}$, the code integrates only from 0 to $x_{\max}$.

In code, `initial_conditions()` uses:

- even: $\psi(0) = 1$, $\psi'(0) = 0$,
- odd: $\psi(0) = 0$, $\psi'(0) = 1$.

Then `build_full_wavefunction()` reflects the half-domain solution to the left side.

This choice makes the algorithm simpler, faster, and more stable. It also lets the states be classified physically as even or odd.

Parity is also useful for the double well because tunneling splitting appears as pairs of even and odd states. The lowest two states are symmetric and antisymmetric combinations of wavefunctions localized in the two wells.

6 Why root finding and bisection?

For each trial energy, the boundary mismatch gives a number:

$$F(E) = \text{mismatch}(E). \tag{12}$$

Allowed eigenvalues satisfy

$$F(E) = 0. \tag{13}$$

So the eigenvalue problem becomes a root-finding problem.

The code first scans energies using `find_brackets()`. It looks for sign changes. If $F(E_1)$ and $F(E_2)$ have opposite signs, then a root is likely between them. Then `bisect_energy()` refines the energy until the interval is small enough.

This is exactly the root-finding logic from the course: bracket a root, then reduce the bracket until the required accuracy is reached.

The code also includes:

- `sample_boundary_mismatch()`,
- `bisection_history()`,
- `sample_inward_decay_mismatch()`,
- `bisection_history_inward_decay()`.

These are mainly for diagnostics and plots. They show that the eigenvalues are not magic numbers. They are roots of a clearly defined numerical function.

7 What each code file does

7.1 `src/potentials.py`

This file defines the physical systems:

- `harmonic_oscillator()`,
- `infinite_square_well_numeric()`,
- `finite_square_well()`,
- `quartic_double_well()`,
- `quartic_oscillator()`,
- `square_barrier()`,
- `double_square_barrier()`.

The purpose is to separate the physics definitions from the numerical solver. This is good design: the solver does not care which potential is chosen. It only needs $V(x)$. That makes the project modular.

The main potentials have clear roles:

- infinite square well: analytic validation,
- harmonic oscillator: analytic validation and inward shooting,
- finite square well: nontrivial finite bound-state system,
- quartic double well: tunneling and energy splitting,
- quartic oscillator: optional extra demonstration,
- single barrier and double barrier: scattering extension.

7.2 src/numerov.py

This is the numerical integration engine. Its main role is to take an x grid, a potential V, and a trial energy E, and produce a trial wavefunction ψ.

The most important functions are:

- `q_from_energy()`,
- `numerov_outward()`,
- `normalize_wavefunction()`,
- `derivative_at_right_edge()`.

Important details:

- `numerov_outward()` includes dynamic rescaling to avoid overflow when a non-eigenvalue trial solution grows exponentially.
- `normalize_wavefunction()` rescales before squaring to avoid overflow.
- `derivative_at_right_edge()` uses a fourth-order stencil when possible.

7.3 src/shooting.py

This is the eigenvalue solver. Its main role is to try energies, integrate with Numerov, check the boundary mismatch, find the correct eigenvalues, and return normalized eigenstates.

The most important functions are:

- `initial_conditions()`,
- `find_brackets()`,
- `bisect_energy()`,
- `solve_symmetric_potential()`,
- `solve_symmetric_potential_inward_decay()`.

Important latest detail: `initial_conditions()` now includes higher Taylor terms so the first Numerov step does not reduce the convergence order. This fixed the square-well convergence slope problem.

7.4 src/analysis.py

This is the scientific validation layer. It contains:

- `exact_square_well_energies()`,
- `exact_harmonic_oscillator_energies()`,
- `relative_error()`,

- `estimate_convergence_slopes()`,
- `convergence_vs_grid()`,
- `convergence_vs_box_size()`,
- `convergence_vs_box_size_fixed_spacing()`,
- `splitting_vs_parameter()`.

This is what turns the code from "it runs" into "it was verified scientifically."

For cases without analytic solutions, such as the quartic double well, the code can use a high-resolution numerical solution as the reference. This is the right approach because there is no exact spectrum to compare with.

7.5 `src/rk4_compare.py`

This is the method-comparison module. It solves the harmonic oscillator using RK4 shooting.

The harmonic oscillator is used because it has exact energies and a smooth potential, so it is a fair comparison.

RK4 works differently from Numerov. Numerov solves the second-order equation directly. RK4 rewrites it as a first-order system:

$$\psi' = \phi, \tag{14}$$
$$\phi' = 2[V(x) - E]\psi. \tag{15}$$

This means RK4 evolves both ψ and ψ' explicitly.

The comparison is useful because it shows the difference between a specialized solver and a general-purpose ODE solver. It also explains why the derivative boundary condition mattered so much. RK4 carries ψ' as a variable, while Numerov needs a finite-difference derivative estimate when enforcing some boundary conditions.

7.6 `src/scattering.py`

This is the scattering extension. It computes:

- transmission T,
- reflection R,
- conservation check $T + R$.

The method is:

1. assume the potential is zero far left and far right,
2. impose a transmitted right-moving wave on the far right,
3. integrate backward through the potential with complex Numerov,

4. decompose the left-side wave into incident and reflected parts.

The main functions are:

- `numerov_outward_complex()`,
- `integrate_from_right()`,
- `decompose_left_asymptotic()`,
- `solve_scattering()`,
- `sweep_scattering()`,
- `find_transmission_peaks()`.

This is not the core bound-state project, but it shows that the same Numerov framework can be extended to scattering and resonant tunneling.

7.7 `src/plotting.py`

This file generates report figures:

- `plot_potential_and_states()`,
- `plot_probability_densities()`,
- `plot_energy_comparison()`,
- `plot_error_curve()`,
- `plot_splitting_curve()`,
- `plot_root_finding_diagnostic()`,
- `plot_scattering_coefficients()`,
- `plot_scattering_potential_and_probability()`,
- `plot_numerov_vs_rk4_errors()`.

These plots let the reviewer visually see the states, convergence, tunneling behavior, scattering behavior, and method comparison.

7.8 `src/experiments.py`

This file connects the solver to the actual project cases. It runs:

- `run_square_well()`,
- `run_harmonic_oscillator()`,
- `run_double_well()`,
- `run_finite_square_well()`,

- `run_quartic_oscillator_demo()`,
- `run_scattering()`.

This file is basically the scientific experiment plan in code.

Important latest double-well additions:

- the double-well plot legend now shows the explicit formula,
- root-finding diagnostics are now generated for the double well,
- double-well convergence versus grid spacing is included,
- double-well convergence versus box size is included,
- double-well convergence uses a high-resolution numerical reference instead of pretending there is an exact analytic solution.

7.9 `scripts/run_solver.py`

This is the entry point. It:

- clears old results,
- runs all experiments,
- runs tests,
- prints a completion message.

It is intentionally lightweight. Most logic lives in `src/`.

7.10 `tests/test_solver.py`

This file checks correctness. It tests:

- normalization,
- derivative stencil accuracy,
- RK4 bracket detection,
- square-well ground-state accuracy,
- square-well convergence order,
- harmonic oscillator energies,
- harmonic oscillator inward shooting,
- double-well splitting,
- scattering probability conservation,
- `run_solver` import behavior.

Important latest regression test: `test_square_well_convergence_order()` requires the first square-well states to show near-fourth-order convergence. This prevents the startup-order bug from quietly returning.

8 What the project demonstrates

The project demonstrates three levels of correctness.

8.1 Level 1: Exact benchmarks

The square well and harmonic oscillator have known exact energies. If the numerical method reproduces them, the solver is credible.

Square well:

- validates the basic outward Numerov shooting method,
- checks node structure and parity,
- checks fourth-order convergence after the improved startup values.

Harmonic oscillator:

- validates inward shooting,
- checks parity and node structure,
- checks domain truncation effects,
- supports the Numerov versus RK4 comparison.

8.2 Level 2: Convergence

The project varies:

- grid spacing h,
- domain size $x_{\max}$.

The goal is to show that the energies stabilize. This addresses truncation error, resolution dependence, finite-domain effects, floating-point limits, and root-finding limits.

A key interpretation point is that if a convergence slope is slightly above 4, this does not mean the method is truly higher than fourth order. It usually means the errors are very small and the log-log fit is being affected by numerical floors, root-finding tolerance, or floating-point limits.

8.3 Level 3: New physics

The double well has nearly degenerate even and odd pairs. This shows tunneling splitting, which is the strongest physics result in the project.

The double-well analysis now includes:

- states and densities,
- energy splitting versus parameter b,
- root-finding diagnostics,
- convergence versus h,
- convergence versus $x_{\max}$,
- a high-resolution reference solution.

9 What is apparent from the lectures and what is beyond them?

9.1 Clearly from the lectures

- ODEs,
- boundary-value problems,
- eigenvalue problems,
- shooting,
- root finding,
- bisection,
- numerical integration,
- numerical differentiation,
- convergence,
- numerical errors,
- documentation.

9.2 Partly from the lectures, but specialized to quantum mechanics

- Numerov method,
- wavefunction normalization,
- parity classification,
- bound-state interpretation,
- inward shooting from a decaying tail,
- tunneling splitting,
- scattering transmission and reflection.

The Numerov idea appears in Pang's Schrödinger equation treatment, but it is more specialized than the main lecture slides.

9.3 Beyond the slides

The most advanced or specialized parts are:

- fourth-order-consistent Numerov startup values,
- high-order derivative stencil at the boundary,
- inward shooting from the forbidden region,
- high-resolution numerical reference for non-analytic potentials,
- RK4 versus Numerov method comparison,
- complex Numerov scattering,
- double-barrier resonant tunneling.

9.4 Not used from the lectures

- advection schemes,
- finite-volume methods,
- limiters,
- Burgers equation,
- Euler equations,
- MHD,
- PIC,
- chaos,
- Monte Carlo,
- FFT,
- matrix eigenvalue solvers.

This is fine. The final project explicitly allows choosing a different physics problem or going beyond class material.

10 The clean explanation for a reviewer

Here is the project summary in reviewer language:

> I am solving the one-dimensional time-independent Schrödinger equation as a boundary-value eigenvalue problem. I discretize space on a uniform grid and use the Numerov method to integrate the second-order ODE for a trial energy. Since only special energies satisfy the required boundary conditions, I use a shooting method. The boundary

mismatch becomes a scalar function of energy, and I find its roots using bracketing and bisection. For symmetric potentials I use parity to reduce the calculation to the half domain, then reconstruct and normalize the full wavefunction. I validate the solver on the infinite square well and harmonic oscillator, study convergence with grid spacing and domain size, and then apply it to a finite square well and quartic double well to analyze nontrivial bound states and tunneling-induced splitting. I also compare Numerov with RK4 for the harmonic oscillator and include scattering through finite and double barriers as a secondary extension.

That is the full logic of the project.